

Part 1: Capacitive In-Memory Processing — Energy Model Implementation and Design Space Exploration

Project: ZigZag-CIMP Extension

Repository: `semron_cimp/` module within `zigzag-cimp`

Based on:

- Paper 1: Demasius, Kirschen & Parkin, "Energy-efficient memcapacitor devices for neuromorphic computing," *Nature Electronics* 4, 748–756 (2021)
 - Paper 2: Demasius, Iowa & Murmann, "A Blueprint for Accurate, Energy-Efficient DNN Inference via Capacitive In-Memory Processing," *IEEE CCMCC* (2025)
-

1. Introduction and Motivation

Modern deep neural network inference is dominated by matrix-vector multiplications (MVMs), where billions of multiply-accumulate (MAC) operations create immense demand for energy-efficient hardware. Processing-using-memory (PUM) architectures address this by embedding computation directly within memory arrays, eliminating the costly data movement of conventional von Neumann designs. Among PUM approaches, capacitive charge-domain techniques have emerged as particularly promising. The key insight is that the charge relationship $Q = C \times V_{in}$ naturally implements multiplication — voltage encodes the input activation, capacitance encodes the stored weight, and the resulting charge is the product. Accumulation occurs physically along the bitline as charges from multiple cells sum according to Kirchhoff's current law.

Paper 1 introduced the *CapRAM* memcapacitive device, which exploits charge shielding between a gate electrode and a readout electrode to achieve high dynamic range with low static power. Fabricated at the microscale and simulated down to 90 nm and 45 nm, the device demonstrated a projected energy efficiency of 29,600 TOPS/W on MNIST when combined with adiabatic energy recovery and workload sparsity. Paper 2 then extended this work with a hardware–software co-design methodology, deriving the system-level constraints that link ADC resolution, bit-slicing strategies, device variation, readout noise, and DNN quantization into a unified design framework for practical inference accelerators.

The `semron_cimp/` module in this repository implements the analytical energy and noise models from these two papers in Python, serving two purposes: reproducing the key figures from Paper 2 as validation, and providing a design space exploration (DSE) tool (`cimp_analyser.py`) that sweeps across the full hardware parameter space to identify optimal array configurations.

2. Theoretical Background

2.1 Capacitive MAC Operation

In a capacitive PUM crossbar, each memory cell stores a weight as a programmable capacitance C between $C_{\min}$ and $C_{\max}$. During inference, an input voltage V_{in} is applied to the word line. The charge generated at each cell is $Q_i = C_i \times V_{\text{in}}$, and the bitline accumulates the total charge $\sum Q_i$, which represents the dot product of the input vector and the weight column. This charge is then converted to a digital value by an ADC.

The critical distinction from resistive PUM is that capacitive readout is fundamentally non-dissipative — energy stored in a capacitor during charging can in principle be recovered during discharging. Paper 1 leverages this through adiabatic circuit designs, achieving approximately 95% energy recovery from the reactive power component.

2.2 Noise and Signal Fidelity

The fundamental noise limit for capacitive readout is kTC noise, where the RMS voltage noise is $V_{\text{rms}} = \sqrt{(kT/C)}$. To achieve reliable LSB detection at the $3\sigma \leq 0.5$ LSB level, oversampling (averaging) is required. The number of averages N_{av} is derived from Paper 2, Equation 42:

$$N_{\text{av}} = 36 \cdot kT \cdot C_{\text{tot}} \cdot (2^{(S_w-1)} - 1)^2 \cdot (2^{(S_x)} - 1)^2 / ((C_{\max} - C_{\min})^2 \cdot V_{\text{in}}^2)$$

where $C_{\text{tot}} = K \cdot C_{\min} + C_{\text{par}} + N_{\text{cap}} \cdot C_{\max}$ includes the parasitic capacitance ($C_{\text{par}} = K \cdot C_{\min}$ as a baseline assumption), the minimum capacitance of all K cells in the column, and the active capacitive cells N_{cap} that contribute to the signal.

This equation reveals several key trade-offs: larger arrays (higher K) require more averages due to increased parasitic capacitance; higher weight precision (S_w) demands finer charge resolution; and a higher ON/OFF ratio ($C_{\max}/C_{\min}$) reduces N_{av} by improving the signal-to-noise ratio.

2.3 ADC Resolution and Bit-Slicing

The ADC resolution B_y determines how many distinct levels the column sum can represent. Under the full-precision guarantee (FPG), the naive requirement for a W4A8 MVM with $K=128$ is $B_y = 19$ bits — far exceeding practical ADC capabilities. Bit-slicing decomposes the 8-bit activation into $L_x = 8$ one-bit slices ($S_x = 1$), reducing the requirement to $B_y = 12$ bits. Paper 2's A2Q+ extension further tightens this to 7–9 bits through slice-wise ℓ_1 -norm constraints on the quantized weights, training the network to respect hardware accumulator limits.

2.4 Energy Model

The total energy per operation consists of two components:

Capacitive thermal energy (E_{cap}): From Paper 2, Equation 43:

$$E_{\text{cap}} = (L_x \cdot N_{\text{av}} \cdot Q_{\text{LSB}} \cdot V_{\text{in}} \cdot 2^{(B_y)}) / K + L_x \cdot N_{\text{av}} \cdot C_{\min} \cdot V_{\text{in}}^2$$

The first term represents the energy for supplying signal charge across all $2^{(B_y)}$ LSB levels, while the second term accounts for "wasted" energy from charging the minimum (non-information-bearing) capacitance of each device.

ADC energy (Walden FOM): The conversion energy scales as $E_{\text{conv}} = \text{FOM}_W \times 2^{(B_y)}$, where $\text{FOM}_W \approx 1$ fJ/conv-step represents state-of-the-art SAR ADC performance. Normalized

per bit-operation, this contributes a constant baseline of approximately 0.25 fJ·b/OP.

2.5 CRITICAL IMPLEMENTATION INSIGHT: The Dual-B_y Resolution Model

⚠ KEY FINDING — This section documents a non-obvious modeling decision that was essential to reproducing Paper 2's energy figures (Figures 12, 13, and 14). The paper does not explicitly state this separation; it was reverse-engineered during implementation and is the only approach that produces curves matching the published results.

Paper 2 uses a single symbol B_y throughout its equations, but a careful reading of the text reveals that the ADC resolution plays two distinct physical roles that operate at different scales:

- Physical B_y (noise domain):** Determines the total capacitance on the bitline and therefore the thermal noise floor. This governs how many charge levels the analog signal must be distinguishable across, including noise margin. In Equation 36, $N_{\text{cap}} = 2^{(B_y)} / (2^{(S_w)} \cdot 2^{(S_x)})$ sets the number of active capacitive cells contributing to the worst-case noise scenario. In Equation 37, the total charge noise q_n depends on $N_{\text{cap}} \cdot C_{\text{max}}$. This B_y must be conservatively high to guarantee the $3\sigma \leq 0.5$ LSB fidelity constraint.
- Signal B_y (energy domain):** Determines how much charge the system actually moves to encode the useful computation result. In Equation 43, the energy scales as $2^{(B_y)}$ because it represents the number of LSB charge quanta that must be physically shuttled through the crossbar. This B_y reflects the actual information content of the dot product, which is lower than the physical noise budget.

The author's likely reasoning is as follows: the physical ADC must be designed with headroom above the signal range to accommodate noise and ensure reliable digitization. In practice, the ADC captures a wider dynamic range (B_{y,phys}) than the signal actually occupies (B_{y,sig}). The difference — 2 bits of headroom in our implementation — represents the noise margin that separates the information-theoretic requirement from the circuit-design requirement.

In the implementation, this is realized as:

```
B_y,phys = 10 + log2(K / 128)    # Used in N_cap and N_av calculations (noise budget)
B_y,sig  = 8  + log2(K / 128)    # Used in Equation 43 energy calculation (signal charge)
```

Why this matters: Using a single B_y for both roles fails to reproduce the paper's figures. If B_y = 10 (physical) is used in the energy equation, E_{cap} is overestimated by 4× because the energy scales with $2^{(B_y)}$. If B_y = 8 (signal) is used in the noise equation, N_{av} is underestimated because the physical array has insufficient noise margin, and the resulting N_{av} values are too small by roughly an order of magnitude. Only the dual-B_y separation produces energy curves (Figures 12, 13, 14) and noise curves (Figures 9, 10) that simultaneously match the paper.

Physical interpretation: The 2-bit gap (factor of 4×) between physical and signal resolution can be understood as follows. The ADC's full-scale range must cover the worst-case column sum

including noise excursions. The signal occupies only 1/4 of this range on average (2 bits = 4× dynamic range headroom). The remaining 3/4 of the ADC's dynamic range is "wasted" on noise margin — but this waste is precisely what makes the $3\sigma \leq 0.5$ LSB guarantee achievable. The energy for moving signal charge, however, only needs to cover the signal portion, not the full ADC range. Hence the energy equation uses the smaller $B_{y,\text{sig}}$ while the noise equation uses the larger $B_{y,\text{phys}}$.

This dual- B_y model is carried consistently through all energy-related scripts

(`Ecap_R_Pops_12.py`), (`Etot_K_Pops.py`), (`Etot_By_Pops.py`) and the DSE tool

(`cimp_analyser.py`). In (`cimp_analyser.py`), lines 67–68 implement this directly, and the downstream calculations carefully route each B_y variant to its correct equation.

2.6 Manufacturing Feasibility

Paper 2 derives the required programming variation for reliable weight storage (Equation 32):

$$\sigma_{C,\text{pgr}} / (C_{\text{max}} - C_{\text{min}}) = 1 / (6\sqrt{K} \cdot (2^{(S_w-1)} - 1))$$

This must remain below the lithographic variation floor of the target technology: 0.71% for dry DUV (180 nm), 0.11% for ArF immersion (22 nm), or 0.062% for SONOS programming accuracy. This constraint directly limits the maximum feasible column length K for a given weight slice precision S_w .

3. Implementation: Figure Reproduction Scripts

The (`semron_cimp/`) folder contains six standalone Python scripts, each reproducing a specific figure from Paper 2. All scripts implement the same core equations but isolate different parameter sweeps for visualization.

3.1 (`limit_7.py`) — Manufacturing Feasibility (Figure 7)

This script plots the required capacitance programming variation $\sigma_{C,\text{pgr}} / (C_{\text{max}} - C_{\text{min}})$ versus column length K for weight slice widths $S_w \in \{1, 2, 3, 4, 5, 6\}$. It implements Equation 32 and overlays three technology thresholds as dashed lines (dry DUV at 0.71%, immersion lithography at 0.11%, SONOS at 0.062%).

The implementation adds a valuable extension beyond the original paper: colored feasibility zones that shade the regions below each technology threshold, and annotated crossover points showing the maximum feasible K for each S_w under immersion lithography. For example, $S_w = 4$ crosses the immersion threshold at $K \approx 468$, while $S_w = 3$ extends to $K \approx 2,550$. Weight slices of $S_w \geq 5$ are infeasible for all $K \geq 128$ under immersion lithography.

3.2 (`Nav_R_K_9.py`) — Number of Averages vs. ON/OFF Ratio (Figure 9)

This script sweeps the ON/OFF ratio from 5 to 100 for seven matrix sizes ($K = 128$ to 8192), computing N_{av} from Equation 42 with fixed parameters $S_w = 4$, $S_x = 1$, $C_{\text{max}} = 1$ fF, $V_{\text{in}} =$

400 mV, $T = 330$ K.

A key implementation detail is the dynamic ADC resolution calculation: $B_y = 10 + \log_2(K/128)$, which increases by one bit each time K doubles. This models the physical reality that larger arrays must resolve finer charge differences.

The secondary y-axis converts N_{av} to wall-clock time assuming 40 ns per averaging cycle. Results confirm Paper 2's finding: for large arrays ($K > 2048$), the ON/OFF ratio must exceed 50 to keep N_{av} below 100–200.

3.3 `Nav_mac_time_10.py` — N_{av} vs. MAC Bitwidth (Figure 10)

This script explores the exponential reduction in N_{av} when the multiplied bitwidth $S_w \times S_x$ decreases. A critical implementation detail is the non-differential rule: $S_w = 1$ is treated identically to $S_w = 2$ because a single-bit weight does not use differential encoding (two physical capacitors), so its noise budget is equivalent to a 2-bit differential weight. This is why the $S_w = 1$ and $S_w = 2$ curves overlap in the plot.

The practical implication is clear: finer input bit-slicing (lower S_x) is always preferable to finer weight slicing because additional input slices increase latency only linearly, whereas additional input bits increase N_{av} exponentially.

3.4 `Ecap_R_Pops_12.py` — Capacitive Thermal Energy (Figure 12)

This script is where the dual- B_y model (Section 2.5) was discovered and validated. Initial attempts to reproduce Figure 12 using a single B_y value consistently failed: a single $B_y = 10$ produced energy curves that were $4\times$ too high (because 2^{10} vs 2^8 in the energy equation), while a single $B_y = 8$ produced N_{av} values that were far too low (the noise budget was too optimistic). Only after separating the resolution into $B_{y,phys} = 10 + \log_2(K/128)$ for the noise path and $B_{y,sig} = 8 + \log_2(K/128)$ for the energy path did the reproduced curves match Paper 2's Figure 12 in both shape and absolute magnitude. The title of the generated figure explicitly documents this: "*Dual B_y Logic: Physical=10, Signal=8 for $K=128$* ".

The calculation proceeds in three stages. First, N_{av} is computed using $B_{y,phys}$ (the conservative noise resolution) to determine how many averaging cycles the physical array requires. Second, Equation 43's energy is evaluated using $B_{y,sig}$ (the signal resolution) to compute how much charge the system actually moves for useful computation. Third, both N_{av} and the signal energy are multiplied together, because the array must move the signal charge N_{av} times to achieve the required SNR.

The energy is normalized to fJ·b/OP (femtojoules per bit-operation) by dividing the total Joule energy by 10^{15} and then by the total bit-operations ($L_x \times S_w = 8 \times 4 = 32$). Results show that smaller arrays achieve higher energy efficiency, and higher ON/OFF ratios consistently reduce E_{cap} across all array sizes.

3.5 `Etot_K_Pops.py` — Total Energy vs. Column Length (Figure 13)

This script also relies on the dual- B_y model (Section 2.5) throughout its energy calculation. It combines the capacitive thermal energy E_{cap} (dashed lines, computed with $B_{y,sig}$) with the

constant Walden FOM baseline of 0.25 fJ·b/OP (solid black line) to show total energy $E_{\text{tot}} = E_{\text{cap}} + E_{\text{ADC}}$ for different ON/OFF ratios across column lengths $K = 128$ to 8192. Without the dual- B_y separation, the dashed E_{cap} curves would sit $4\times$ too high, destroying the crossover behavior between ADC-dominated and thermal-dominated regimes that is the figure's central insight.

The crossover behavior is clearly visible: for small arrays ($K < 500$), the Walden FOM dominates; for large arrays ($K > 2000$), the thermal limit of the capacitive device becomes dominant, especially at lower ON/OFF ratios. This identifies the fundamental scaling challenge: larger arrays offer better parameter density but worse energy efficiency per operation.

3.6 `Etot_By_Pops.py` — Total Energy vs. ADC Resolution (Figure 14)

This script fixes $K = 8192$ and sweeps the signal ADC resolution $B_{y,\text{sig}}$ from 10 to 14 bits. Critically, it maintains the 2-bit headroom relationship by computing $B_{y,\text{phys}} = B_{y,\text{sig}} + 2$ for the noise/ N_{av} calculation (see Section 2.5). This means as the user reduces the ADC target resolution, both the signal energy and the noise budget scale down in lockstep, preserving the physical consistency of the model. The resulting curves show the characteristic $2^{(2B_y)}$ Schreier FOM behavior. It demonstrates that reducing B_y by just 2 bits (from 14 to 12) yields a nearly $8\times$ reduction in energy per operation — the core motivation for Paper 2's A2Q+ quantization-aware training approach.

4. Implementation: The CIMP Analyser DSE Tool

4.1 Architecture

The `cimp_analyser.py` file implements the `CIMP_Analyzer` class, which encapsulates the complete energy, latency, and feasibility model in a single design space exploration engine. The class has two core methods:

`get_max_feasible_K(Sw, tech)` — Computes the maximum manufacturable column length for a given weight precision and lithography technology, implementing the inverse of Equation 32. This is used as a pruning filter to skip physically impossible configurations during the sweep.

`evaluate_configuration(K, r, Sw, M, Lx)` — Computes all performance metrics for a single array configuration, returning a dictionary with latency (ns), throughput (TOPS), total energy (pJ), energy efficiency (fJ/bOP and POPS/W/b), and energy-delay product (EDP).

4.2 Parameter Space

The DSE sweeps five dimensions simultaneously:

Parameter	Description	Default Range	Paper Link
K	Column length (rows)	128 - 8192 (step 128)	Crossbar dimension
r	ON/OFF ratio (C_max/C_min)	5 - 100 (step 5)	Device quality
S_w	Weight slice width (bits)	1 - 6	Quantization precision
M	Number of columns	128 - 8192 (step 128)	Output parallelism
L_x	Input slices	4, 8, 16, 32	= input_precision / S_x

The full exploration generates up to $64 \times 20 \times 6 \times 64 \times 4 \approx 31$ million configurations, though the feasibility pruning based on `get_max_feasible_K` eliminates a large fraction early.

4.3 Key Formulas Implemented

The `evaluate_configuration` method chains the following calculations:

- ADC resolution:** $B_{y,phys} = 10 + \log_2(K/128)$; $B_{y,sig} = 8 + \log_2(K/128)$
- Parasitic model:** $C_{min} = C_{max}/r$; $C_{par} = K \cdot C_{min}$; $N_{cap} = 2^{(B_{y,phys})} / (2^{(S_{w,eff})} \cdot 2^{(S_x)})$
- Averaging (Eq. 42):** $N_{av} = \max(36 \cdot kT \cdot C_{tot} \cdot (2^{(S_{w,eff}-1)} - 1)^2 \cdot (2^{(S_x)} - 1)^2 / ((C_{max} - C_{min})^2 \cdot V_{in}^2), 1)$
- Latency:** $t_{MAC} = L_x \cdot N_{av} \cdot t_{avg}$ (where $t_{avg} = 40$ ns)
- Energy (Eq. 43):** $E_{signal} + E_{waste}$, normalized to fJ/bOP
- ADC energy:** $E_{ADC} = 0.25 \cdot (4/S_w)$ fJ/bOP (scaled for weight precision)
- Derived metrics:** $POPS/W/b = 1/E_{tot}$; $EDP = E_{tot} \times \text{latency}$; $TOPS = \text{total_bit_ops} / t_{MAC}$

4.4 Output Rankings

The DSE produces four ranked leaderboards optimized for different deployment scenarios:

- Pure Energy Efficiency** — Maximizes POPS/W/b (edge devices, battery-constrained)
- Pure Speed** — Minimizes latency (real-time inference)
- Overall Balance** — Minimizes EDP (Pareto-optimal energy-delay trade-off)
- Massive Throughput** — Maximizes TOPS (data center, batch inference)

Each ranking uses a multi-level sort with tie-breaking criteria to produce unambiguous orderings.

4.5 Link to ZigZag Framework

The code is extensively annotated with `[ZIGZAG LINK]` comments that map each CIMP parameter to its ZigZag equivalent. The key correspondences are: K maps to the D1 (or D2)

dimension size in the accelerator YAML; the energy per MAC from `evaluate_configuration` feeds into the `energy_cost` of ZigZag's multiplier array; and the latency determines the effective clock period. The `capram/` folder demonstrates this integration end-to-end, with `capram_array.py` subclassing ZigZag's `ImcArray` to override the SRAM-based energy model with Paper 1's SPICE-calibrated values.

5. Validation and Results

5.1 Figure Reproduction Accuracy

All six reproduced figures show qualitative and quantitative agreement with Paper 2's original plots.

The single most important implementation finding was the dual-B_y resolution model (Section 2.5). The paper writes Equation 43 with a single B_y, but the published energy curves can only be reproduced when two separate resolutions are used: a physical B_{y,phys} for the noise/averaging path and a signal B_{y,sig} (2 bits lower) for the energy path. This is not stated explicitly anywhere in Paper 2's text — it was discovered by systematic trial and error during implementation. The likely explanation is that the authors, coming from a circuit design background, implicitly understood that the ADC's physical dynamic range includes noise headroom beyond the signal range, and did not consider this worth stating because it is standard practice in ADC design. However, for anyone implementing the equations from the paper, this distinction is absolutely critical: without it, the energy figures are off by 4× and the noise figures are internally inconsistent. The fact that the 2-bit headroom produces simultaneous agreement across all three energy-dependent figures (12, 13, 14) and all three noise-dependent figures (7, 9, 10) strongly suggests this is the correct interpretation rather than a coincidental curve fit.

5.2 Key Findings from the DSE

The analyser confirms Paper 2's central conclusions:

- **ADC resolution is the dominant energy bottleneck.** At small array sizes ($K \leq 256$), the Walden FOM baseline contributes over 70% of total energy, making ADC design the primary optimization target.
- **ON/OFF ratio has diminishing returns.** Beyond an ON/OFF ratio of ~ 50 , further improvement yields only marginal N_{av} reduction due to the saturation caused by the growing $N_{cap} \cdot C_{max}$ term in the total capacitance.
- **Bit-slicing trades latency for precision.** Each halving of S_x doubles the number of compute cycles but halves the required ADC resolution, yielding a net energy benefit.
- **Manufacturing feasibility constrains the design space.** Under immersion lithography, 4-bit weight slices limit K to approximately 468 rows, while 2-bit slices allow K up to $\sim 8,000$.

5.3 Capacitive vs. Resistive Advantage

The extended analysis (`capram/extended_analysis.py`) validates Paper 2's theoretical derivation that capacitive devices are at least $2\times$ more energy efficient than purely resistive (thermal-noise-limited) devices, and $15.4\times$ more efficient than shot-noise-limited devices (such as ReRAM), at the same ADC resolution. This physics-level advantage is compounded by the possibility of adiabatic energy recovery, which is inapplicable to resistive readout.

6. Code Quality and Design Decisions

The implementation makes several pragmatic engineering choices worth noting:

- **Temperature at 330 K** (rather than the standard 300 K) adds thermal margin, reflecting realistic chip operating conditions.
 - **Non-differential rule** ($S_w = 1$ treated as $S_w = 2$) correctly implements the physical constraint that single-bit weights cannot use differential encoding.
 - **Hardware minimum $N_{av} = 1$** prevents the model from predicting sub-unity averages at very small arrays with high ON/OFF ratios.
 - **Progress bar with ETA** in the DSE loop provides practical usability for sweeps that can take minutes.
 - **Feasibility pruning** skips entire K branches when they exceed the manufacturing limit, providing significant speedup without loss of completeness.
-

7. Summary

The `semron_cimp/` module provides a complete, validated implementation of the capacitive in-memory processing energy model from the SEMRON/Nature Electronics 2021 and IEEE CCMCC 2025 papers. The six figure-reproduction scripts serve as unit tests for the underlying physics equations, while the `cimp_analyser.py` DSE tool extends this into a practical hardware configuration optimizer. Together, they form the analytical foundation for the ZigZag-CIMP integration demonstrated in the companion `capram/` module, bridging the gap between device-level physics and system-level architecture exploration.